

Mahidol University
Wisdom of the Land

Mahidol University



ITDS241 Web Technologies and Applications

Object-Oriented Programming in JavaScript

Instructor: Asst. Prof. Wudhichart Sawangphol, Ph.D., SFHEA
Email: wudhichart.saw@mahidol.ac.th

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Class Objectives

- Students will be able
  - To create and use functions in JS
  - To explain the concept of Object-oriented programming in JS
  - To explain the concept of the html document object model
  - To apply the OOP and DOM concept to the JS programming

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University

Wisdom of the Land

Mahidol University

Functions

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Why use functions?

- There are some main reasons:
  - **Code Reusability**: Functions allow you to write reusable blocks of code.
  - **Modularity**: Functions help break down complex problems into smaller, manageable parts.
  - **Code Organization**: Functions provide a structured approach to organizing your code.
  - **Code Maintainability**: When you need to modify or fix a specific functionality, you can focus on the relevant function without affecting other parts of the codebase.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Functions

- Function
- Constructor Function
- First Class Function
- High-order Function
- Callback Function
- Arrow Function

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Functions (cont.)

- There are two types of function:
  - Functions that do not return values
  - Functions that do return values

```
function showMessage() {
alert('Hello everyone!');
}
```

Not return value

```
function sum(a, b) {
return a + b;
}
```

Return value

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Functions with Parameters (Arguments)

- Function in JS support passing arbitrary data to it. The data is called **Parameters** or **Arguments**.

```
function showMessagefromuser(from, text) {
alert(from + ': ' + text);
}
```

- Parameters can be set **default value**.

```
function showMessagefromuser(from, text="no message")
{
alert(from + ': ' + text);
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Arrow functions

- Arrow function has simple and concise syntax for creating functions.

```
let func = function (arg1, arg2, ...argN) {
return expression;
};
```



```
let func = (arg1, arg2, ...argN) => expression
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Arrow functions

- Examples:

```
let sum = (a,b) => a + b;
```

- No parameter:

```
let sayHello = () => alert("Hello!");
```

- Multiple line:

```
let subtract = (a, b) => {
let result = a - b;
return result;
};
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Constructor Functions

- Normally, we might want to make many objects that have the same set of properties in OOP since they can be proceed in a similar way without error.
- Constructor functions offer a mechanism to create objects with identical sets of properties.
- Constructor functions can be created in the same way as normal function.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Constructor Functions (cont.)

- They are called with the **`new`** operator
- In constructor function, the keyword **`this`** is used to refer to the object being created.
- Every newly created object is assigned with a consistent set of properties
- We can initialise the value to properties using the parameters of the function.
- We can add public (using this) and private (using var, let) functions in constructor functions

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Constructor Functions (cont.)

- For example,

```
function Person(firstName, lastName, age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.getInfo = function(){
                return this.firstName + " " + this.lastName + " Age is "
                        + this.age;
        };
}
```

```
var person1 = new Person("Wudhichart", "Sawangphol", 30);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Constructor Functions (cont.)

- Interestingly, the properties of an object can be added on the fly.
- A constructor function can inherit another constructor function.

- See example.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

First Class Functions

- A programming language is said to have First-class Functions when functions in that language are treated like any other variable.
  - A function can be assigned to a variable (Anonymous function or Expression function).
  - It can be passed as a parameter to other functions.
  - It can be returned by another function.
  - It can be stored in some data structures.
- This can be used to great effect in a programming paradigm called functional programming.
- See example

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

First Class Functions (cont.)

- Assign a function to a variable

```
//Create an anonymous function
var function2 = function (wide) {
var area;
area = Math.pow(wide, 2);
return area;
};
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

First Class Functions (cont.)

- Pass a function as an argument

```
function flexible(fOperation, operand1,
operand2) {
var result = fOperation(operand1, operand2);

return result;
}
```

```
let sumFunction = function (num1, num2) {
return num1 + num2;
};

output = flexible(sumFunction, 3, 4);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

First Class Functions (cont.)

- Return a function

```
const calculateAreaofSquare2 = function (wide) {
var area;
area = Math.pow(wide, 2);
return () => {
console.log(`calculateAreaofSquare2: ${area}`);
};
};

let areaOfSquare = calculateAreaofSquare2(4);
areaOfSquare();
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

First Class Functions (cont.)

- Storage in a Data Structure

```
let myFuncs = [calculateAreaofSquare];
console.log(`calculateAreaofSquare: ${myFuncs[0](5)}`);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

High-order Function

- High-order functions are functions that can take other functions as arguments or return functions as their results.

```
function calculateAreaofSquare(wide) {
var area;
area = Math.pow(wide, 2);
return area;
}
```

```
let squareWide1 = [2, 7, 9, 10];
console.log("Map: " +
squareWide1.map(calculateAreaofSquare));
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

High-order Function (cont.)


```
const evenNumbers = squareWide1.filter((num) => num % 2 === 0);
console.log(`evenNumbers: ${evenNumbers}`);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Callback Functions

- The primary purpose of a callback function is to provide a mechanism for a program to respond to events or actions that occur asynchronously or unpredictably.
- APIs (Application Programming Interfaces) are implemented libraries of objects and their methods.
- Sometimes, API functions will require a function as a parameter. You are expected to code this function with a parameter list specified in the API documentation. This function is called Callback functions.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Callback Functions (cont.)


```
function greet(){
alert('Howdy!');
}
setTimeout(greet, 2000);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables

- Functions are like office departments.
- Scope (https://vimeo.com/202865903)
  - JavaScript variables are lexically scoped except `var`.
  - For variables that are declared outside of any function, their scope is all code associated with the web page. (Global variable)
  - For variables that are declared in a function, their scope is the entire function. (Local variable)
- Note! Variables should have the smallest scope.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- Example:

```
<script type="text/javascript">
if (true) {
const myName = "Wudhichart";
console.log(myName);
}
</script>
```

- For this script, `myName` is said to be block-scoped to the `if` statement.
- `myName` can only be accessed from within the block.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- If we try to access `myName` outside of the block, we get a `ReferenceError` because it is out of scope.

```
<script type="text/javascript">
if (true) {
const myName = "Wudhichart";
console.log(myName);
}

console.log(myName); // Uncaught ReferenceError: myName is not defined
</script>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- Scope of `let`, `const`, and `var`
  - `let` and `const` create block-scoped variables.
  - `var` create a variable that is scoped to their nearest function or the global scope.

```
function displayFullname() {
if (true) {
var myFullname = "Wudhichart Sawangphol";
}
console.log(myFullname);
}

displayFullname();
```

There is no error.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- **Global** and Module Scope

Global Scope

```
<script type="text/javascript">
const myFullname = "Wudhichart Sawangphol";
</script>
<script type="text/javascript">

// Global and Module Scope
console.log(myFullname);
</script>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- Global and **Module** Scope
- Each module also has its own scope.

Module Scope

```
<script type="module">
const jidapa = "Jidapa Kraisangka";
</script>
<script type="text/javascript">

// Global and Module Scope
console.log(jidapa); // Uncaught ReferenceError: jidapa is not defined
</script>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Scope and Lifetime of Variables (cont.)

- Lifetime
  - Global variables: the time from they are declared until the web page is closed
  - Local variables: the time from a function, where they are declared in, is called until the function is terminated
- If the names of global variable and local variable are the same, the local variable is considered.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Closures in JavaScript

- A closure is a function combined with references to the variables defined outside of it.
- Closures maintain the variable references, which allow functions to access variables outside of their scope.

```
//Closures
function displayFullname(name) {
return () => console.log(name);
}

const printFullname = displayFullname("Wudhichart Sawangphol");

printFullname();
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

Object-oriented programming (OOP)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

What is OOP?

- OOP: Object-oriented programming
- Same concepts as we represent things in real life e.g., cars, persons, bank accounts, etc.
- Define the data type of a data structure and the type of functions that can be applied to the data structure

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Basic OOP: Class

- **Class** is a template or a model for our objects consisted of properties and functions.

Properties

Class: Instructor

```
firstName
lastName
subjects
```



```
sayHello("Hey! I am [firstName].
          Nice to meet you!!");
teach("I teach [subjects].");
```

Methods

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Basic OOP: Object

- **Object** is a class filled in with “values”. When an object is instantiated from a class, it has all properties and methods defined in the class

Class: Instructor



```
firstName
lastName
subjects

sayHello("Hey! I am [firstName].
          Nice to meet you!!");
teach("I teach [subjects].");
```

"Instantiation"

Object: swudhichart



```
firstName: "Wudhichart"
lastName: "Sawangphol"
subjects: ["WWW","JS"]

sayHello("Hey! I am Wudhichart.
          Nice to meet you!!");
teach("I teach WWW,JS.");
```

Object: kjidapa



```
firstName: "Jidapa"
lastName: "Kraisangka"
subjects: ["HTML","JS:OOP"]

sayHello("Hey! I am Jidapa.
                Nice to meet you!!");
teach("I teach HTML, JS:OOP.");
```

Object: ppisit



```
firstName: "Pisit"
lastName: "Paiwattana"
subjects: ["jQuery","CSS"]

sayHello("Hey! I am Pisit.
          Nice to meet you!!");
teach("I teach jQuery,CSS.");
```

COPYRIGHT © 2017-2025 FACULTY OF ICT, MAHIDOL UNIVERSITY All Rights Reserved.

Define a class in JS

In JS, we can define a class (class declaration) by the following syntax

```
class className {
      constructor(prop1,prop2,…){
            /*Property definition
            and assignment*/
      }

      /* Method definition */
      method1(){/* do something */}
      method2(){/* do something */}
}
```

The **constructor method** is **called** each time the object is initiated.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Instructor Class

Note: Do not forget to put inside `<script type="application/javascript">…</script>`

```
class Instructor {
        /* Constructor */
        constructor(_firstName, _lastName, _subjects)
        {
                /* Properties */
                this.firstName = _firstName;
                this.lastName  = _lastName;
                this.subjects  = _subjects;
         }

        /* Methods*/
        sayHello() {
                return "Hey! I'm " + this.firstName + ". Nice to meet you!!!";
         }
        teach() {
                return "I teach " + this.subjects + ".";
         }
}
```

Class: Instructor



```
firstName
lastName
subjects

sayHello("Hey! I am [firstName].
                Nice to meet you!!");
teach("I teach [subjects].");
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Create an object in JS

- Class must be defined before the object is created
- Use new to call the class constructor to initiate an object with the specified values.

```
let obj = new className(prop1,prop2,…);
```

- To access a property in an object,

```
/* Get a value of a property */
console.log(obj.propName);
/* Assign a new value to a property */
obj.propName = newValue;
```

- To call a method,

```
obj.methodName();
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Object of `Instructor` class

Note: the `Instructor` class must be defined before initiating an object

```
/* Initiate an object of the Instructor class */
let jkSubjects = ["HTML","OOP:JS"];
let kjidapa = new Instructor("Jidapa", "Kraisangka",
          jkSubjects);


/* Display firstName and lastName in the console */
console.log(kjidapa.firstName + " " + kjidapa.lastName);

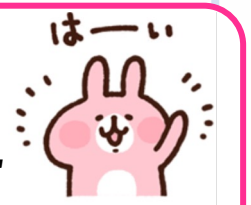
/* Change firstName from "Jidapa" to "Aj. Pa" */
kjidapa.firstName = "Aj. Pa";
console.log(kjidapa.firstName);

/* Call sayHello() and teach() and
   display in the console */
console.log(kjidapa.sayHello());
console.log(kjidapa.teach());
```

Object: kjidapa

```
firstName: "Jidapa"
lastName: "Kraisangka"
subjects:["HTML","JS:OOP"

sayHello("Hey! I am Jidapa.
          Nice to meet you!!");
teach("I teach HTML, JS:OOP.");
```



OUTPUT

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Inheritance and Polymorphism

- Objects may have some different properties and methods besides general one from their superclass.
- A subclass inherits all the properties and methods of a superclass (inheritance) but can also have it own unique properties and methods (polymorphism)
- Use `extends` to create a subclass inherited from a superclass

```
class subclassName extends superclassName {
        constructor(cprop1,cprop2,…, scprop1,scprop2,…){
                /* Properties inherited from the superclass */
                super(cprop1,cprop2,…);
                /* Properties of the subclass */
                this.scprop1 = scprop1;
                …
        }
        /* Superclass's methods also inherited from the superclass */
        /* Only define methods that are unique for the subclass */
        subclassMethod1(){/* do something here */} …
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: UnivInstructor Class

Note: the `Instructor` class must be defined before defining the `UnivInstructor` subclass

```
/* Superclass:Instructor, Subclass:UnivInstructor*/
class UnivInstructor extends Instructor {
        /* Constructor */
        constructor(_firstName, _lastName, _subjects,
                    _university, _campus)
        {
                /* Properties from the superclass */
                super(_firstName,_lastName,_subjects);

                 /* Properties only in the subclass */
                this.university  = _university;
                this.campus  = _campus;
         }
        /* Methods also inherited from the superclass */
         getUniversity() {
                return "My office is at " + this.university + ", " +
                        this.campus +".";
         }
}
```

Class: UniInstructor

firstName
lastName
subjects
university
campus

sayHello()
teach()
getUniversity()



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Object of UnivInstructor Class


```
/* Instantiate an object of the UnivInstructor class */
let jkSubjects = ["HTML","OOP:JS"];
let kjidapa2 = new UnivInstructor("Jidapa","Kraisangka",
               jkSubjects,"Mahidol University","Salaya");


/* Display firstName and university in the console */
console.log(kjidapa2.firstName +"@"+ kjidapa2.university);



/* All methods from the superclass are inherited */
console.log(kjidapa2.sayHello());
console.log(kjidapa2.teach());

/* Call getUniversity() method */
console.log(kjidapa2.getUniversity());
```

Object: kjidapa2




```
firstName: "Jidapa"
lastName: "Kraisangka"
subjects: ["HTML","JS:OOP"]

university: "Mahidol University"
campus: "Salaya"

sayHello()
teach()
getUniversity()
```

OUTPUT

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Object of `UnivInstructor` Class (cont.)


```
/* What if we try adding an undefined property */
console.log(kjidapa2.nickName);

kjidapa2.nickName = "Pa";
console.log(kjidapa2.nickName);


/* What if we try calling an undefined function */
kjidapa2.sayMyName();

kjidapa2.sayMyName = function () {
        return "I am " + this.firstName +
                " or " + this.nickName;
}
console.log(kjidapa2.sayMyName());
```

Object: kjidapa2




```
firstName: "Jidapa"
lastName: "Kraisangka"
subjects: ["HTML","JS:OOP"]

university: "Mahidol University"
campus: "Salaya"
nickName: ??

sayHello()
teach()
getUniversity()
sayMyName() ??
```

OUTPUT

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University

Wisdom of the Land

Mahidol University

Document Object Model (DOM)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

DOM: Document Object Model

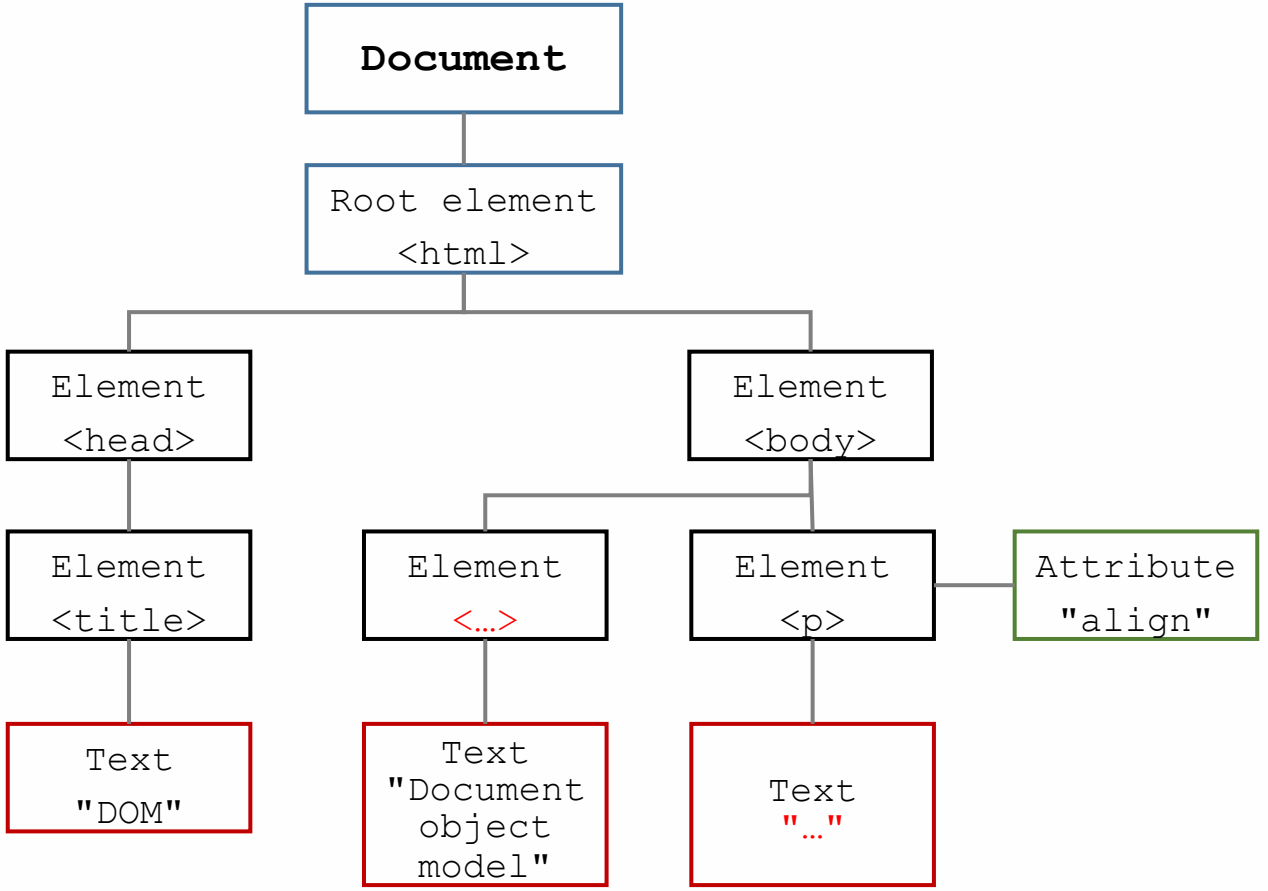
- The DOM is a W3C (World Wide Web Consortium) standard.
- DOM is a programming interface for web documents.
- It allows programs and scripts to dynamically access and update the content, structure, and style of a document
- The DOM is separated into different parts:
  - Core DOM - a standard set of objects for any structured document.
  - XML DOM - a standard set of objects for XML documents.
  - HTML DOM - a standard set of objects for html documents

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

HTML DOM

- A `html` page is constructed as a tree of objects

```
<html>
  <head>
    <title>DOM</title>
  </head>
  <body>
    <h1>Document object model</h1>
    <p align="left">
       A HTML document is
       constructed as
       a tree of objects
    </p>
  </body>
</html>
```



Ref: https://www.w3schools.com/js/js_htmldom.asp

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

HTML DOM (cont.)

- All `html` elements along with their texts and attributes can be accessed through the DOM.
- The contents can be modified or deleted, and new elements can be created.
- The HTML DOM is platform and language independent. It can be used by any programming language like Java, JavaScript, and Python.

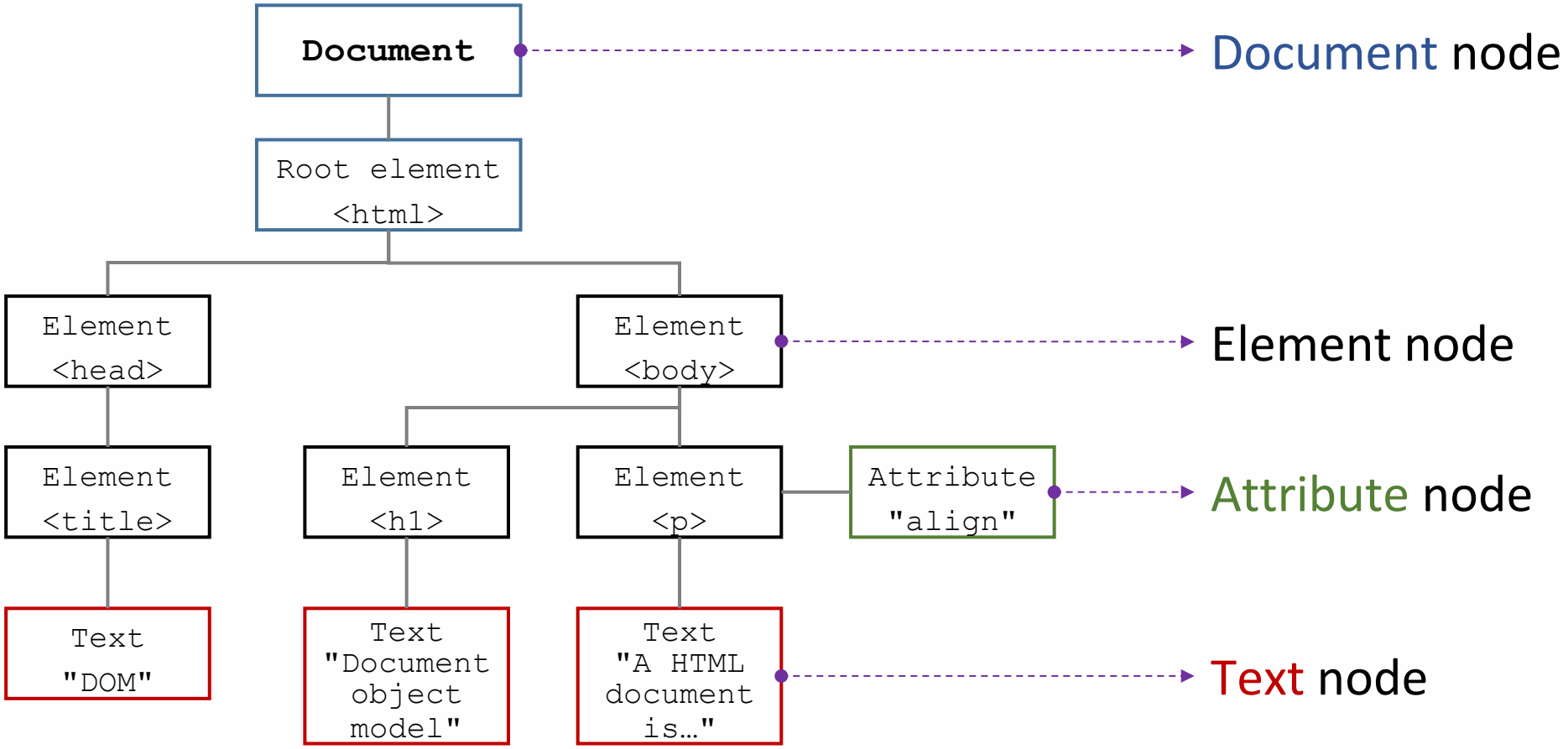
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Structure of HTML DOM

- Everything in an html document is a node.
- Nodes
  - The entire document is a document node
  - Every HTML tag is an element node
  - The texts contained in the HTML elements are text nodes
  - Every HTML attribute is an attribute node
  - Comments are comment nodes
- Nodes have a **hierarchical** relationship to each other

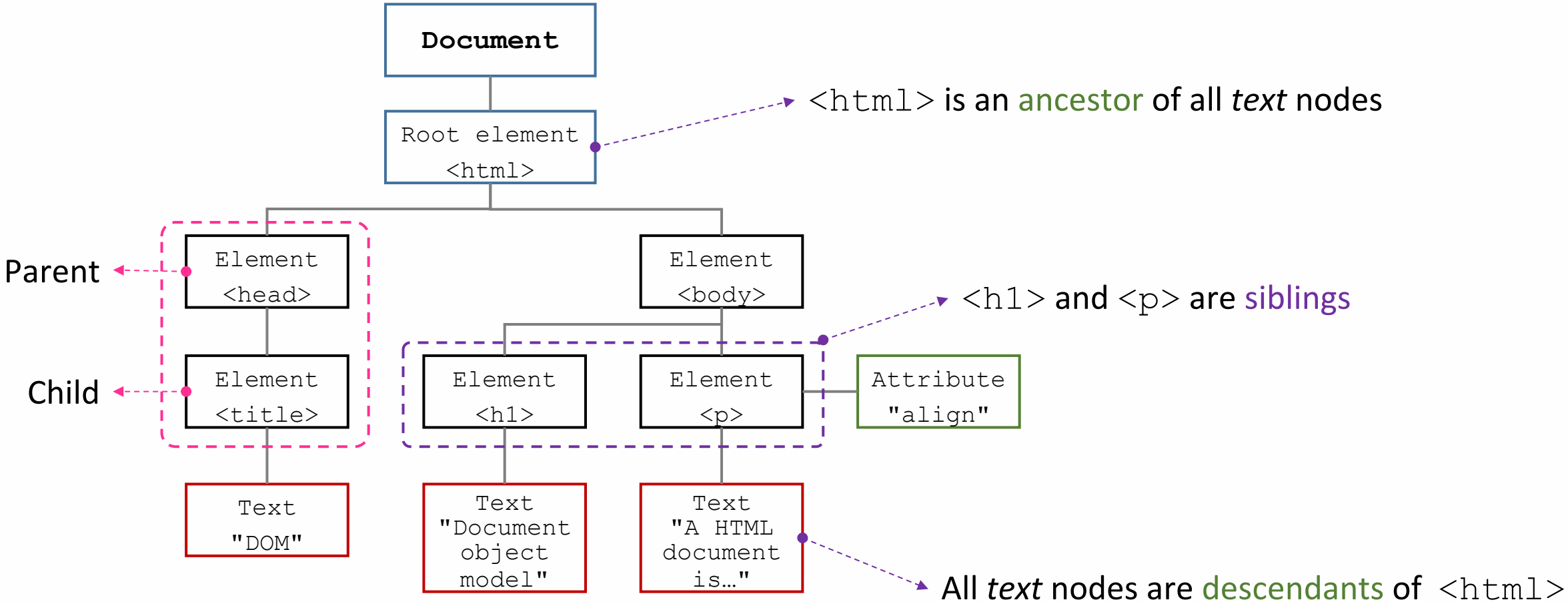
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Structure of HTML DOM



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Structure of HTML DOM



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Document object with JS

- JS can manipulate all parts of the html document including elements, attributes, events, CSS in the page
- Many methods are provided in the document object.
  - (more details at https://developer.mozilla.org/en/docs/Web/API/Document)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Summary of `Document` Methods

Methods	Descriptions
`document.getElementById()`	Returns the element that has the ID attribute with the specified value
`document.getElementsByClassName()`	Returns a NodeList containing all elements with a specified class name
`document.getElementsByTagName()`	Returns a NodeList containing all elements with the specified tag name
`document.write()`	Write html expressions or JavaScript code to a document
`document.writeln()`	Same as write(), but adds a newline character after each statement

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Summary of `Document` Methods

Methods	Descriptions
`document.querySelector()`	Returns the first Element within the document that matches the specified selector, or group of selectors
`document.querySelectorAll()`	returns a static NodeList representing a list of the document's elements that match the specified group of selectors

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Element object

- Element objects are parts of a `html` document, also implemented as nodes

```
<h1 style="color:blue;">DOM Lesson one</h1>
```

- Element object consists of
  - An opening tag
  - Attribute (s)
  - Text
  - A closing tag

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Summary of `Element` methods/properties

Methods/Properties	Descriptions
element.innerHTML	Sets or gets the HTML syntax describing the element's descendants.
node.textContent	Returns / Sets the textual content of an element and all its descendants.
node.nodeValue	Returns / Sets the value of the current node
node.childNodes	Returns a live NodeList containing all the children of this node.
node.firstChild	Returns a Node representing the first direct child node of the node, or null if the node has no child.
node.getRootNode()	Returns the context object's root which optionally includes the shadow root if it is available.

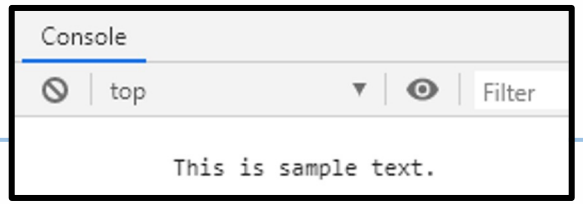
COPYRIGHT © 2017-2024 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Access a Node/Element

- Use `document.getElementById()` to retrieve the element that has the ID attribute with the specified value

```
let selected_element = document.getElementById("ID");
```

```
<html>
    <body>
        <div id="div_content">
            This is sample text.
        </div>
        <script type="application/javascript">
            /* Use this method to get the selected node with ID */
            let divNode = document.getElementById("div_content");
            /* After getting a node,we can access/manipulate them */
            console.log(divNode.textContent);
        </script>
    </body>
</html>
```



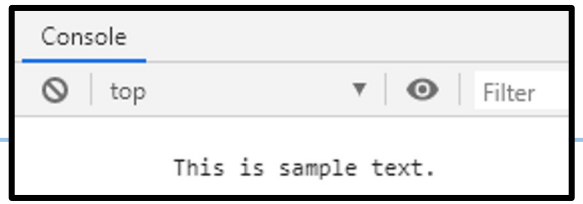
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Use CSS selectors to Access a Node/Element

- Use `document.querySelector()` to retrieve the first element that matches the specific selector or group of selectors

```
let selected_element = document.querySelector(selectors);
```

```
<html>
    <body>
        <div id="div_content">
            This is sample text.
        </div>
        <script type="application/javascript">
            /* Use this method to get the selected node with ID */
            let divNode = document.querySelector("#div_content");
            /* After getting a node,we can access/manipulate them */
            console.log(divNode.textContent);
        </script>
    </body>
</html>
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Access a collection of Nodes/Elements

- Use `document.getElementsByClassName()` to retrieve a collection of nodes with the specified name in the format of array.

```
let selected_elements = document.getElementsByClassName("name");
```

```
<div id="div_content" style = "background-color: yellow;"> This is sample text.</div>
<span class="colors">R</span>
<span class="colors">G</span>
<span class="colors">B</span>
<script type="application/javascript">
        let divNode = document.getElementById("div_content");
        /* Use this method to get a collection of nodes */
        let colorNodes = document.getElementsByClassName("colors");
        /* After getting a node list, we can access/manipulate them */
        let output ="Color: ";
        for (var i = 0; i < colorNodes.length; i++) {
            output += colorNodes[i].textContent + " ";
        }
        divNode.textContent = divNode.textContent + output;
</script>
```



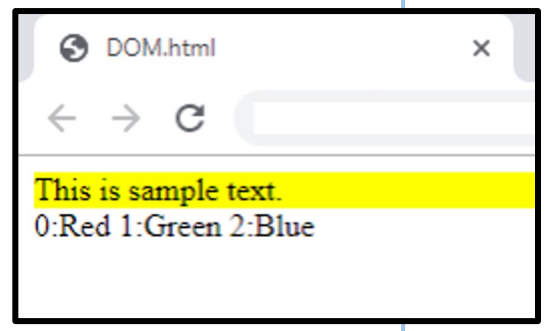
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Access a collection of Nodes/Elements

- Use `document.getElementsByTagName()` to retrieve a collection of nodes with the specified tags in the format of array.

```
let selected_elements = document.getElementsByTagName("tag");
```

```
<div id="div_content" style = "background-color: yellow;"> This is sample text.</div>
<span class="colors">Red</span>
<span class="colors">Green</span>
<span class="colors">Blue</span>
<script type="application/javascript">
        let divNode = document.getElementById("div_content");
        /* Use this method to get a collection of nodes */
        let colorNodes = document.getElementsByTagName("span");
        /* After getting a node list, we can access/manipulate them */
        for (var i = 0; i < colorNodes.length; i++) {
           colorNodes[i].textContent = i + ":" + colorNodes[i].textContent;
        }
</script>
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

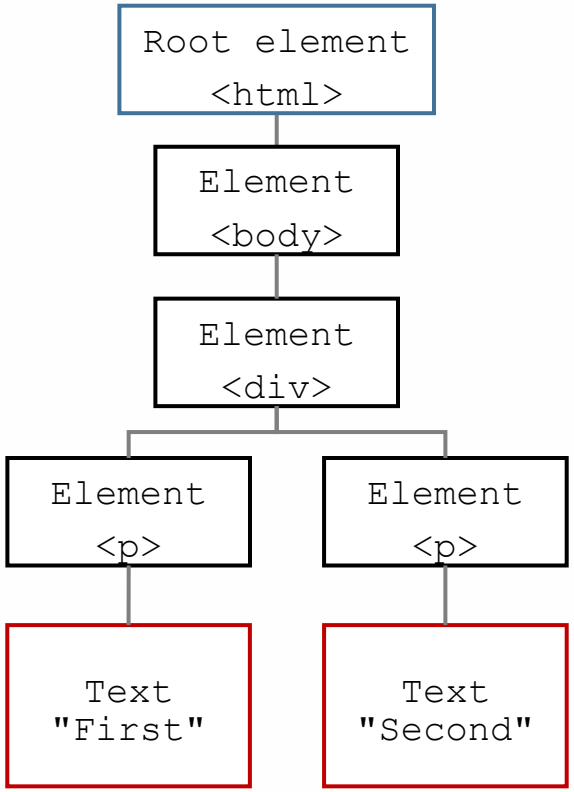
Add a node/element

- Use `createElement()` with `createTextNode()` to create a node
- Use `appendChild()` to add a new node to the last child of its parent

```
<div id="div_demo"></div>
<script type="application/javascript">
        /* Use this method to get the <div> element */
        let divDemo = document.getElementById("div_demo");

        /* Add <p> with contents in the "div_demo"
        innerHTML gets all contents between the <div>...</div> tag */
        divDemo.innerHTML = "<p>First</p>";

        /* Add another paragraph <p> by another method */
        var p2 = document.createElement("p");
        var txtP2 = document.createTextNode("Second");
        /* Add p2 to the <div> */
        p2.appendChild(txtP2);    // Append text node to <p>
        divDemo.appendChild(p2); // Append <p> to <div>
</script>
```



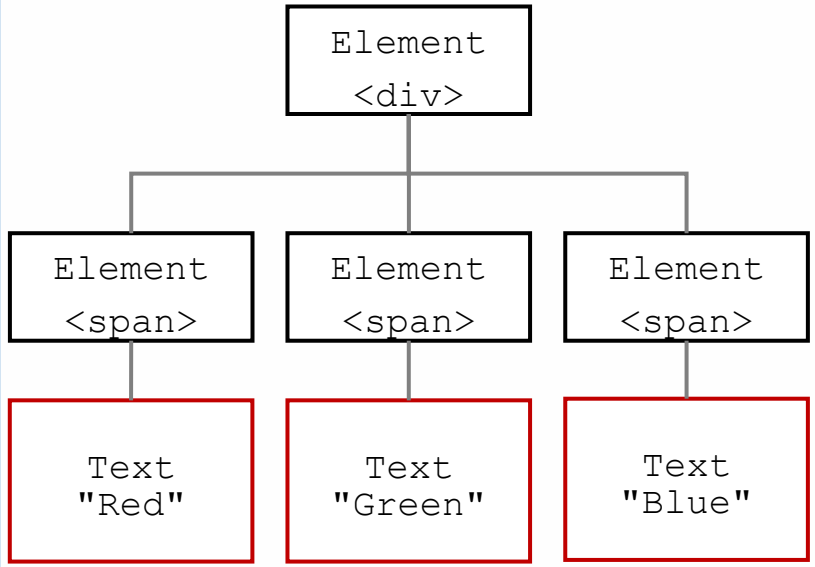
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Remove a node/element

- Use `removeChild()` to delete the *selected node*. Make sure you find the right child first

```
<div id="div_color">
        <span id="color_R">Red</span>
        <span id="color_G">Green</span>
        <span id="color_B">Blue</span>
</div>

<script type="application/javascript">
        /* Use this method to get the <div> element */
        let divColor = document.getElementById("div_color");
        /* Use this method to get the <span>
        element to be removed */
        let colorG = document.getElementById("color_G");
        /* Delete the child from parent: Bye bye! */
        divColor.removeChild(colorG);
</script>
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Access Node/Element attributes

- After getting a node, use `setAttribute("attribute", "value")` to set a node attribute.

```
<div id="div_color">
        <span id="color_R">Red</span>
        <span id="color_G">Green</span>
        <span id="color_B">Blue</span>
</div>

<script type="application/javascript">
        /* Use this method to get the <span> element */
        let spanColor = document.getElementsByTagName("span");

        /* Set a "class" attribute to all <span> nodes */
        for(var i=0; i<spanColor.length; i++){
           spanColor[i].setAttribute("class","colors");
        }
</script>
```

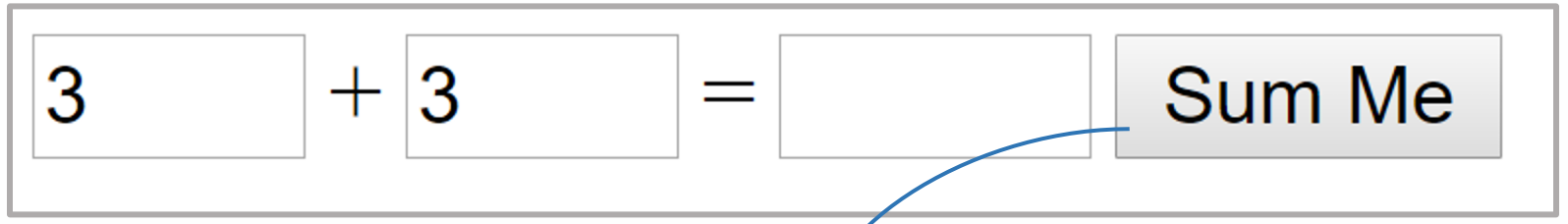
```
▼<div id="div_color">
   <span id="color_R" class="colors">Red</span>
   <span id="color_G" class="colors">Green</span>
   <span id="color_B" class="colors">Blue</span>
 </div>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Events

- An `html` event can be something the browser does, or something a user does.

Example



onclick

```
function sum(){
/* Sum up two numbers */
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Summary of Events

Event	Description
`onabort`	Fires when image transfer has been interrupted by user.
`onchange`	Fires when a new choice is made in a select element, or when a text input is changed, and the element loses focus.
`onclick`	Fires when the user clicks using the mouse.
`ondblclick`	Fires when the mouse is double clicked.
`onfocus`	Fires when a form element gains focus.
`onkeydown`	Fires when the user pushes down a key.
`onkeypress`	Fires when the user presses then releases a key.
`onkeyup`	Fires when the user releases a key.
`onload`	Fires when an element and all its children have loaded.
`onsubmit`	Fires when a form is submitted.
`onunload`	Fires when a page is about to unload.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Summary of Events (cont.)

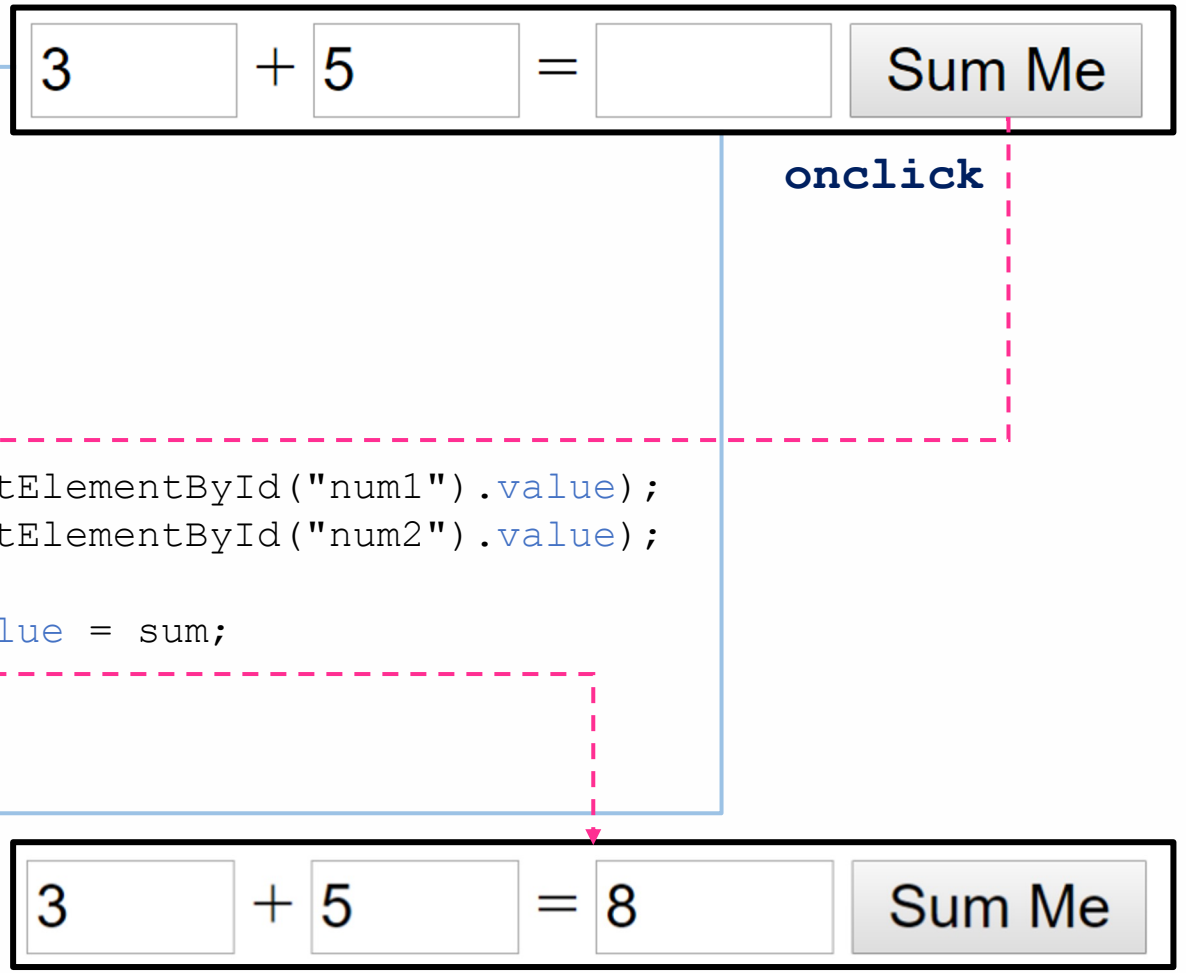
Event	Description
`onmousedown`	Fires when a mouse button is pressed down.
`onmousemove`	Fires when the mouse moves.
`onmouseout`	Fires when the mouse leaves an element.
`onmouseover`	Fires when the mouse enters an element.
`onmouseup`	Fires when a mouse button is released.
`onreset`	Fires when a form resets (i.e., the user clicks a reset button).
`onresize`	Fires when the size of an object changes (i.e., the user resizes a window or frame).
`onselect`	Fires when a text selection begins (applies to `input` or `textarea`).

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Sum up Two numbers


```
<body>
        <input type="text" size="2" id="num1"> +
        <input type="text" size="2" id="num2"> =
        <input type="text" size="3" id="sum">
        <button onclick="sum()">Sum Me</button>

        <script type="application/javascript">
            function sum(){
                var num1 = parseFloat(document.getElementById("num1").value);
                var num2 = parseFloat(document.getElementById("num2").value);
                var sum = num1+num2;
                document.getElementById("sum").value = sum;
            }
        </script>
</body>
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Example: Playing with Mouse Events


```
<body>
        <div id="demo" style="background-color:red; color:white; font-family:sans-
        serif" onmouseover="myMouseOver()" onmouseout="myMouseOut()" onmousedown=
        "myPress()" onmouseup="myRelease()">
                This is me!!!!
        </div>
        <script type="application/javascript">
            function myMouseOver(){
                let divDemo = document.getElementById("demo");
                divDemo.innerHTML = "I put my mouse OVER this!";}
            function myMouseOut(){
                let divDemo = document.getElementById("demo");
                divDemo.innerHTML = "This is me!!!!";}
            function myPress(){
                let divDemo = document.getElementById("demo");
                divDemo.innerHTML = "I am pressing...";
                divDemo.style.backgroundColor = "green";}
            function myRelease(){
                let divDemo = document.getElementById("demo");
                divDemo.innerHTML = "This is me!!!!";
                divDemo.style.backgroundColor = "red";}
        </script>
</body>
```

This is me!!!!

I put my mouse OVER this!

This is me!!!!

I am pressing...

This is me!!!!

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Events and JS

- An event can be added using JS.
- There is a method called **addEventListener**. This method is used to add an event to an element.

```
let selected_element = document.getElementById("ID");
selected_element.addEventListener(”click”, MyClick)
```

- All events can be found here.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Today's Wrap Up

- JS is one of the OOP, which allow you to create user-defined classes or use the pre-defined class/methods
- A `html` page is constructed from a tree of elements.
- With the DOM concept, you can freely manipulate all elements using the predefined methods/properties.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

References

- W3Schools. (2025). *JavaScript Tutorial*. https://www.w3schools.com/js/
- MDN. (2025). *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.